游戏助手服务接口说明（MMR 服务）

目录

1 游戏助手服务接口说明（Multimodal Retrieval MMR 服务） **2**

2 更新日志 **2**
2.1 2025-12-17 . . . 2

3 MMR 服务 **2**
3.1 配置文件 . . . 3
3.2 接口 `/mmr/service/enable` . . . 3
3.2.1 Method `Post` . . . 3
3.2.2 Content-Type `application/json` . . . 3
3.2.3 参数 . . . 3
3.2.4 返回 . . . 3
3.2.5 例子 . . . 4
3.3 接口 `/mmr/service/init` . . . 4
3.3.1 Method `Post` . . . 4
3.3.2 Content-Type `application/json` . . . 4
3.3.3 参数 . . . 4
3.3.4 返回 . . . 4
3.3.5 例子 . . . 5
3.4 接口 `/mmr/service/list` . . . 5
3.4.1 Method `Get` . . . 5
3.4.2 Content-Type `application/json` . . . 5
3.4.3 参数 . . . 5
3.4.4 返回 . . . 5
3.4.5 例子 . . . 5
3.5 接口 `/mmr/service/delete` . . . 5
3.5.1 Method `Post` . . . 6
3.5.2 Content-Type `application/json` . . . 6
3.5.3 参数 . . . 6
3.5.4 返回 . . . 6
3.5.5 例子 . . . 6
3.6 接口 `/mmr/record/list` . . . 6
3.6.1 Method `Post` . . . 6
3.6.2 Content-Type `application/json` . . . 6
3.6.3 参数 . . . 6
3.6.4 返回 . . . 6
3.6.5 例子 . . . 7

3.7 接口 /mmr/record/delete . . . 7
3.7.1 Method Post . . . 7
3.7.2 Content-Type application/json . . . 7
3.7.3 参数 . . . 7
3.7.4 返回 . . . 7
3.7.5 例子 . . . 8
3.8 接口 /mmr/record/insert . . . 8
3.8.1 Method Post . . . 8
3.8.2 Content-Type multipart/form-data . . . 8
3.8.3 参数 . . . 8
3.8.4 返回 . . . 8
3.8.5 例子 . . . 9
3.9 接口 /mmr/service/query . . . 9
3.9.1 Method Post . . . 9
3.9.2 Content-Type multipart/form-data . . . 9
3.9.3 参数 . . . 9
3.9.4 返回 . . . 10
3.9.5 例子 . . . 10
3.10 接口 /mmr/service/build . . . 11
3.10.1 Method Post . . . 11
3.10.2 Content-Type application/json . . . 11
3.10.3 参数 . . . 11
3.10.4 返回 . . . 11
3.10.5 例子 . . . 12

1 游戏助手服务接口说明（Multimodal Retrieval MMR 服务）

编辑日期 2025-12-17 。
游戏助手服务版本 2025-12-17 。

2 更新日志

2.1 2025-12-17

- 初版。
- 提供 MMR 服务 接口说明。

3 MMR 服务

该服务提供简易的图、文以及图文混合存储，支持混合任务 Embedding 搜索。
每个 Instance 即一个实例，代表一个数据库，对应于一个具体游戏。
分为本地数据以及网络数据。本地数据来源游戏过程中玩家自定义数据，网络数据来源于网站游戏攻略数据。
每条 Record 即一条记录，以键值对的形式存储，键的类型是字符串，值的类型可以是字符串、数字。针对长文本，可以指定其生成 Embedding 用于搜索。
每条 Record 也可以附上单张图片，用于图相关的混合检索。
多模态检索参数，详见 配置文件 。

3.1 配置文件

配置文件在 `config` 目录下，名为 `mmr.json` 。

```
{
    "mmr": {
        "enable": true,
        "model_name": "gme",
        "device": "iGPU",
        "gme": {
            "model_path": "models/mmr/gme",
            "min_pixels": 3136,
            "max_pixels": 1003520,
            "max_ratio": 200,
            "size_factor": 28,
            "max_length": 1800,
            "instruction": "You are a helpful assistant.",
            "vectorstore_dim": 1536,
            "normalize": true
        }
    }
}
```

配置项说明：
1. enable：Bool 类型。是否启用 MMR 服务。
2. model_name：String 类型。指定使用的模型名称，当前支持 `"gme"` 。该名称同时作为键名，对应下方同名子对象中的模型参数。
3. device：String 类型。用于指定模型加载的设备，如 `"iGPU"` 。
4. gme：Dict 类型。`model_name` 为 `"gme"` 时对应的模型参数。
- model_path：String 类型。模型文件所在路径。
- min_pixels：Int 类型。图像缩放的最小像素数。
- max_pixels：Int 类型。图像缩放的最大像素数。
- max_ratio：Int 类型。图像宽高比的最大允许值。
- size_factor：Int 类型。图像缩放的尺寸因子，宽高必须为该值的倍数。
- max_length：Int 类型。图文混合输入序列的最大 token 总长度（包含系统提示、图像占位 token 和用户文本）。
- instruction：String 类型。默认的系统指令提示词。
- vectorstore_dim：Int 类型。向量库的维度。
- normalize：Bool 类型。是否对 Embedding 向量进行 L2 归一化。

3.2 接口 `/mmr/service/enable`

启停或查询 MMR 服务。

3.2.1 Method `Post`

3.2.2 Content-Type `application/json`

3.2.3 参数

1. `enable` ：Bool 类型。`null` 表示查询，`true` 启用，`false` 停用。

3.2.4 返回

格式：

```
{
    "code": " 返回代码",
    "msg": " 失败原因",
    "data": {
        "mmr": bool 类型。是否启用 MMR 服务。
    }
}
```

返回的 code ：
1. ok ：成功。不会出现 msg 。enable 为 null 时会返回 data ；enable 为 true 或 false 时不会返回 data 。
2. fail ：失败。会有 msg 给出失败原因。

3.2.5 例子

```
# 查询 MMR 服务是否启用

import json
import requests

url = f"http://127.0.0.1:9190/mmr/service/enable"
data = { "enable": None }
resp = requests.post(url, data=json.dumps(data))
resp_data = resp.json()
""" resp_data
{
    "code": "ok",
    "data": {
        "mmr": true
    }
}
"""
```

```
# 启用 MMR 服务

import json
import requests

url = f"http://127.0.0.1:9190/mmr/service/enable"
data = { "enable": True }
resp = requests.post(url, data=json.dumps(data))
resp_data = resp.json()
""" resp_data
{
    "code": "ok"
}
"""
```

3.3 接口 /mmr/service/init

创建 MMR 服务实例。

3.3.1 Method Post

3.3.2 Content-Type application/json

3.3.3 参数

1. instance_id ：String 类型。实例 ID。

3.3.4 返回

格式：

```
{"code": " 返回代码", "msg": " 失败原因"}
```

返回的 code ：
1. ok ：成功。不会出现 msg 。
2. fail ：失败。会有 msg 给出失败原因。

3.3.5 例子

```
# 创建实例，实例 ID 为`test_instance`

import json
import requests

instance_id = "test_instance"
url = f"http://127.0.0.1:9190/mmr/service/init"
data = { "instance_id": instance_id }
resp = requests.post(url, data=json.dumps(data))
resp_data = resp.json()
# resp_data: {"code": "ok"}
```

3.4 接口 /mmr/service/list

列出所有实例。

3.4.1 Method Get

3.4.2 Content-Type application/json

3.4.3 参数

无

3.4.4 返回

格式：

```
{
    "code": " 返回代码",
    "msg": " 失败原因",
    "data": {
        "instance_ids": [
            " 实例 ID 列表"
        ]
    }
}
```

返回的 code ：

1. ok ：成功。不会出现 msg 。
2. fail ：失败。会有 msg 给出失败原因。不会出现 data 。

3.4.5 例子

```
# 列出所有实例

import requests

url = f"http://127.0.0.1:9190/mmr/service/list"
resp = requests.get(url)
resp_data = resp.json()
# resp_data: {"code": "ok", "data": {"instance_ids": ["test_instance1"]}}
```

3.5 接口 /mmr/service/delete

删除实例。

3.5.1 Method `Post`

3.5.2 Content-Type `application/json`

3.5.3 参数

1. `instance_id` : String 类型。实例 ID。

3.5.4 返回

格式：

```
{"code": " 返回代码", "msg": " 失败原因"}
```

返回的 `code` :

1. `ok` : 成功。不会出现 `msg` 。
2. `fail` : 失败。会有 `msg` 给出失败原因。

3.5.5 例子

```
# 删除实例`test_instance`

import json
import requests

instance_id = "test_instance"
url = f"http://127.0.0.1:9190/mmr/service/delete"
data = { "instance_id": instance_id }
resp = requests.post(url, data=json.dumps(data))
resp_data = resp.json()
# resp_data: {"code": "ok"}
```

3.6 接口 `/mmr/record/list`

列出指定实例下的所有 Record。

3.6.1 Method `Post`

3.6.2 Content-Type `application/json`

3.6.3 参数

1. `instance_id` : String 类型。实例 ID。

3.6.4 返回

格式：

```
{
    "code": " 返回代码",
    "msg": " 失败原因",
    "data": {
        "record_ids": [
            "Record 列表"
        ]
    }
}
```

返回的 `code` :

1. `ok` : 成功。不会出现 `msg` 。
2. `fail` : 失败。会有 `msg` 给出失败原因。不会出现 `data` 。

3.6.5 例子

```
# 列出所有 Record，
# 实例 ID 为`test_instance`。

import json
import requests

instance_id = "test_instance"
url = f"http://127.0.0.1:9190/mmr/record/list"
data = { "instance_id": instance_id }
resp = requests.post(url, data=json.dumps(data))
resp_data = resp.json()
""" resp_data
{
  "code": "ok",
  "data": {
    "record_ids": [
        {
            "document_id": "<Record ID>",
            "content": "<Embedding string>",
            "metadata": {
                "text": "<Text>",
                "info": {
                    "key1": "value1",
                    "key2": "value2"
                }
            }
        }
    ]
  }
}
"""
```

3.7 接口 `/mmr/record/delete`

从指定实例中，根据入库数据的 id 删除所有相同 id 的记录。

3.7.1 Method `Post`

3.7.2 Content-Type `application/json`

3.7.3 参数

1. `instance_id` ：String 类型。实例 ID。
2. `record_id` ：String 类型。指定 id 删除符合 id 的每条记录。

3.7.4 返回

格式：

```
{"code": " 返回代码", "msg": " 失败原因"}
```

返回的 `code` ：

1. `ok` ：成功。不会出现 `msg` 。
2. `fail` ：失败。会有 `msg` 给出失败原因。

3.7.5 例子

```
# 删除 Record,
# 实例 ID 为`test_instance`,
# Record ID 为`test_record`

import json
import requests

instance_id = "test_instance"
record_id = "test_record"
url = f"http://127.0.0.1:9190/mmr/record/delete"
data = { "instance_id": instance_id, "record_id": record_id }
resp = requests.post(url, data=json.dumps(data))
resp_data = resp.json()
# resp_data: {"code": "ok"}
```

3.8 接口 /mmr/record/insert

插入一条 Record。

3.8.1 Method Post

3.8.2 Content-Type multipart/form-data

3.8.3 参数

1. data ：String 类型。JSON 格式的字符串，其内容包含以下参数。
   - instance_id ：String 类型。实例 ID。
   - text ：String 类型。默认为空字符串。图像的关键描述。
   - info ：Dict[String, Any] 类型。Record 数据内容，以键值对的形式存储，键的类型是字符串，值的类型可以是字符串、数字。用于返回图、文或者 url 等面向用户的数据
2. filedata ：Byte 类型。默认空。图片文件的 Byte 数据。

格式：

```
{
    "data": {
        "instance_id": "Instance ID",
        "text": "This is the keyword information paired with the image.",
        "info": {
            "web":"http://xxx.com",
            "guide": " 游戏攻略",
            "datapath":" 要给用户展示的图或者视频的路径，如果没有就空字符串"
        }
    }
}
```

3.8.4 返回

格式：

```
{
    "code": " 返回代码",
    "msg": " 失败原因",
    "data": {
        "text": " 入库文字信息",
        "info": " 附加信息"
```

```
    }
}
```

返回的 `code` ：
1. `ok` ：成功。不会出现 `msg` 。
2. `fail` ：失败。会有 `msg` 给出失败原因。不会出现 `data` 。

3.8.5 例子

```python
# 插入一条 Record，
# 实例 ID 为`test_instance`。
# Record ID 由系统根据内容自动生成（sha256）。

import json
import requests

doc = {
    "web": " 数据出处",
    "guide": " 攻略文字",
    "datapath": " 用于检索返回后，需要展示的视频图像网址等数据"
}

instance_id = "test_instance"
text_key = "Describe the boss in the image."
images = [r"C:\Users\Public\Pictures\0001.jpg",]
url = f"http://127.0.0.1:9190/mmr/record/insert"
data = {
    "instance_id": instance_id,
    "text": text_key,
    "info": doc,
}
data = {'data': json.dumps(data, ensure_ascii=False)}
files = [ ("filedata", open(image, 'rb')) for image in images ]
resp = requests.post(url, data=data, files=files)
resp_data = resp.json()
""" resp_data
{
    "code": "ok",
    "data": {
        "text": " 入库文字信息",
        "info": { " 附加信息键值对" }
    }
}
"""
```

3.9 接口 `/mmr/service/query`

通过混合图文查询 Record，图或文可以空，但是不能同时为空。

3.9.1 Method `Post`

3.9.2 Content-Type `multipart/form-data`

3.9.3 参数

1. `data` ：String 类型。JSON 格式的字符串，其内容包含以下参数。
    - `instance_id` ：String 类型。实例 ID。

- `text` ：String 类型。查询文本内容。
- `topk` ：Int 类型。默认 1。仅返回分数最高的 K 个 Record。必须大于 0。
- `threshold` ：Float 类型。默认 0.01。筛选结果的阈值，仅返回分数超过 `threshold` 的 Record。取值范围 `0.0 ~ 1.0` 。

2. `filedata` ：Byte 类型。图片文件的 Byte 数据。

格式：

```
{
    "data": {
        "instance_id": "Instance ID",
        "text": " 查询内容",
        "topk": 1,
        "threshold": 0.01,
    }
}
```

3.9.4 返回

格式：

```
{
    "code": " 返回代码",
    "msg": " 失败原因",
    "data": [
        [
            [
                分数,
                {
                    "document_id": " 记录的唯一标识（sha256）",
                    "content": "embedding 向量字符串",
                    "metadata": {
                        "text": " 入库文字信息",
                        "info": { " 附加信息键值对" }
                    }
                }
            ]
        ]
    ]
}
```

返回的 `code` ：
1. `ok` ：成功。不会出现 `msg` 。
2. `fail` ：失败。会有 `msg` 给出失败原因。不会出现 `data` 。

`data` 为查询结果列表，每条结果是一个二元组 `[相似度得分，Document]` 。
`Document` 包含以下字段：
- `document_id` ：String 类型。记录的唯一标识，由入库内容的 sha256 生成。
- `content` ：String 类型。入库时的 embedding 向量字符串。
- `metadata` ：Dict 类型。入库时附带的元数据，包含 `text` 和 `info` 。

3.9.5 例子

```
# 查询获胜的 Record。
# 实例 ID 为`test_instance`。

import json
```

```
import requests

pic_1 = r"C:\Users\Public\Pictures\0001.jpg"

url = f"http://127.0.0.1:9190/mmr/service/query"
data = {
    "instance_id": "test_instance",
    "topk": 1,
    "threshold": 0.01
}
data = {'data': json.dumps(data, ensure_ascii=True)}
files = [ ("filedata", open(pic_1, 'rb')) ]
resp = requests.post(url, data=data, files=files)
resp_data = resp.json()
""" resp_data
{
    "code": "ok",
    "data": [
        [
            [
                0.867,
                {
                    "document_id": " 使用 content 的 sha256 计算的索引值",
                    "content": "[ 0.0056525   0.0222293   0.01626055 ... -0.01423412 -0.00432305 -0.01084446]
                    "metadata": {
                        "text": " 入库文字信息",
                        "info": {
                            "web": " 数据出处信息",
                            "guide": " 攻略信息",
                            "datapath": " 检索后需要展示的多模态信息，比如攻略视频、攻略图等"
                        }
                    }
                }
            ]
        ]
    ]
}
"""
```

3.10 接口 /mmr/service/build

将 faiss 实例数据库和 meta 数据保存至磁盘。
如果数据库不会自动保存，需要调用此接口立即保存数据库。

3.10.1 Method Post

3.10.2 Content-Type application/json

3.10.3 参数

1. instance_id ：String 类型。实例 ID。

3.10.4 返回

格式：

```
{"code": " 返回代码", "msg": " 失败原因"}
```

返回的 code ：
1. ok ：成功。不会出现 msg 。

2. `fail` ：失败。会有 `msg` 给出失败原因。

3.10.5 例子

```
# 保存实例数据库

import json
import requests

instance_id = "test_instance"
url = f"http://127.0.0.1:9190/mmr/service/build"
data = { "instance_id": instance_id }
resp = requests.post(url, data=json.dumps(data))
resp_data = resp.json()
# resp_data: {"code": "ok"}
```